

WEEK 4 - INTERN EXERCISE TASKS

1. JavaScript — Student Result Processor

Objective: Process structured student data using intermediate JavaScript array and object operations.

```
const students = [  
  { id: 1, name: "Ali", marks: [70, 80, 65] },  
  { id: 2, name: "Sara", marks: [90, 85, 88] },  
  { id: 3, name: "John", marks: [45, 55, 50] }  
];
```

Create functions to:

- calculate average marks for every student
 - add a grade property
 - return students who passed
 - sort students by average marks
 - find the highest-scoring student
 - calculate overall class average
 - do not modify the original array
-

2. JavaScript — Inventory Management

Objective: Build a small inventory utility that safely manages product data.

```
const products = [  
  { id: 1, name: "Keyboard", price: 50, stock: 10 },  
  { id: 2, name: "Mouse", price: 25, stock: 4 },  
  { id: 3, name: "Monitor", price: 200, stock: 7 }  
];
```

Implement functions to:

- add a new product
 - update product stock
 - update product price
 - remove a product by id
 - search products by name
 - return products below a given stock quantity
 - calculate total inventory value
-

3. JavaScript — Async User Data Loader

Objective: Use asynchronous JavaScript to retrieve and combine related API data.

- Use a public API such as JSONPlaceholder.
- Create async function `getUserDetails(userId)`.
- Fetch the selected user and then fetch posts belonging to that user.
- Combine the results into one object containing user and posts.
- Create async function `getMultipleUsers(ids)` and use `Promise.all()` to load several users.
- Use `async/await` and `try/catch`.
- Handle invalid IDs and failed requests without crashing the program.
- Return a meaningful error result when a request fails.

Task 4 — JavaScript: Shopping Cart Management System

Objective: Create a Shopping Cart Management System using JavaScript only.

Implement the following functionality:

- `addToCart(productId, quantity)` — Add a product to the cart.
- If the product already exists in the cart, increase its quantity instead of creating a duplicate.
- Do not allow quantity to exceed available stock.
- `removeFromCart(productId)` — Remove a product completely.
- `updateQuantity(productId, quantity)` — Change the quantity of an existing cart item.
- `getCartTotal()` — Calculate the total price of all cart items.
- `applyDiscount(percent)` — Apply a percentage discount to the total.
- `getCartSummary()` — Return a complete summary.